

Optimal End-to-End Architecture for Simulacra (Educational Wargaming Platform)

Executive Summary

Simulacra requires a robust architecture that supports two distinct modes: **Experience Mode** (a real-time 2D simulation with game-like visuals) and **Reflection Mode** (a post-game analytics suite with heavy UI for debriefing). After in-depth analysis, the recommended end-to-end architecture is a **hybrid** approach that combines a game-optimized front end (canvas/WebGL for graphics with an HTML/React overlay for UI), a Node.js real-time backend using Colyseus for session management, a stateless Python “Engine” service for game adjudication via simple HTTP calls, and a reliable persistence layer (PostgreSQL) with careful consistency practices for capturing game logs. This design balances fast development (using proven frameworks) with future scalability and aligns with Simulacra’s dual goals:

- **Frontend:** Use a **hybrid Phaser+React** client. Phaser (or similar 2D engine) will render the interactive simulation canvas, while React handles HUD overlays, menus, and the Reflection Mode UI. This provides high-performance graphics and smooth gameplay, alongside flexible, responsive UI for charts and logs.
- **Realtime Backend:** Use **Node.js + Colyseus** to manage game rooms, WebSocket communication, and state synchronization between players. Colyseus offers built-in real-time gaming abstractions (rooms, state delta sync, client sync) that accelerate development and easily handle Simulacra’s scale (a few hundred concurrent users).
- **Game Engine Service:** Keep the **Python Adjudication Engine** as a separate microservice accessed via stateless HTTP API. This simplicity is sufficient given Simulacra’s turn-based rounds (~30s latency tolerance). gRPC or streaming can be deferred until needed; HTTP/REST is easier to implement and maintain for an MVP.
- **Data & Consistency:** Treat **PostgreSQL as the source of truth** for game outcomes by logging every round’s events (actions, options, consequences) in an append-only history. Ensure consistency by writing to the database *before* broadcasting results to players, avoiding “dual-write” race conditions. Use Redis sparingly for ephemeral needs (distributed locks or round timers) if running multiple server instances. The stored history enables full **replay, decision tree visualization, and counterfactual analysis** in Reflection Mode, as planned.

Overall, the proposed architecture closely aligns with the team’s current hypothesis, but refines it with an emphasis on reliability and clarity. The hybrid front end and Colyseus backend are confirmed as optimal for rapid development and smooth user experience. The stateless engine via HTTP is deemed sufficient for MVP, with a migration path to gRPC/streaming if low-latency streaming of AI narratives ever becomes necessary. Finally, a careful data consistency strategy (inspired by the transactional outbox pattern) will safeguard the integrity of game logs for the Reflection Mode. The following sections provide a layer-by-layer analysis, trade-offs, and recommendations for implementing this architecture in an MVP.

Frontend Architecture: Hybrid Canvas + DOM for Dual-Mode UX

Recommendation: Implement a **hybrid front-end** that uses a canvas/WebGL-based game engine for the simulation visuals, combined with standard React/HTML components for UI overlays and the

analytics interface. This approach leverages the strengths of both rendering paradigms – high-performance graphics for Experience Mode and flexible, accessible UI for Reflection Mode.

- **Canvas/WebGL for Simulation:** A 2D game engine like **Phaser 3** is ideal for the immersive “Experience Mode.” Phaser provides a full suite of game development features (rendering, animation, physics, input, asset loading) while internally using efficient WebGL rendering (via Pixi.js) for smooth performance ¹ ². It offloads low-level graphics work to the GPU, enabling rich visuals (like an RTS or Civ-style map) and smooth animations of game elements. By using a game engine, the team won’t need to manually manage redraws or performance optimizations – Phaser’s game loop and rendering pipeline handle that, letting developers focus on game mechanics and visuals.
- **React/DOM for UI and Overlay:** Layering a React-based HUD on top of the canvas allows building complex user interfaces (menus, buttons, tooltips, charts, forms) with standard web techniques. Using the DOM for these static or UI elements is not only easier for web developers, it can **improve performance** by keeping UI outside the game’s main rendering loop ³. DOM elements (built with HTML/CSS) are ideal for the Reflection Mode’s needs: data tables, graphs, and interactive analysis tools can be implemented with existing libraries (D3.js, Chart.js, etc.) instead of reinventing them in a canvas. React’s component model and state management will help organize these features and ensure the UI is maintainable and responsive. Notably, **Phaser and React can be integrated:** an official template demonstrates how Phaser manages the canvas while React manages external UI, enabling “complex UI elements external to the Phaser canvas” in a seamless way ⁴. In practice, the game canvas would be a fullscreen or main panel, and React renders HUD elements in an overlay `<div>` positioned on top (e.g., showing scores, timers, action selection menus, etc.).
- **DOM vs Canvas Trade-offs:** A **DOM-only** approach (e.g. using just React/DOM for everything, even game graphics) was considered but is suboptimal for Simulacra’s dynamic visuals. The DOM can handle simple animations or small grids, but for rich 2D graphics with potentially many moving parts, it becomes inefficient. Canvas/WebGL can draw thousands of moving sprites or effects far more efficiently than manipulating equivalent DOM elements ⁵. Conversely, a **Canvas-only** approach (drawing all UI, text, and charts on the canvas) would complicate development and compromise flexibility – every button or chart would require custom rendering and lose the benefits of HTML semantics and accessibility. The hybrid model gives the best of both: **Canvas for rendering-intensive game visuals, DOM for UI controls and text** ⁶ ⁷. This split also aligns with best practices from HTML5 game development: use the DOM as a “free built-in UI framework” for static interface elements, and reserve canvas for the actual game graphics ⁶.
- **Phaser vs. Alternatives:** **Phaser 3** is recommended given its maturity and ease of use – it’s specifically designed for 2D browser games and even novice game developers pick it up quickly ⁸ ¹. It already integrates Pixi.js internally for fast rendering, and it will handle cross-platform concerns (desktop and mobile browsers) out of the box. An alternative is **Pixi.js** alone, which is “extremely fast” as a pure rendering engine but provides no game logic or UI tools ². Using Pixi would mean writing more custom game-loop code and possibly needing additional libraries for things like physics or keyboard input ⁹. Since Simulacra’s game mechanics (turn-based, grid/strategic actions) are not overly physics-heavy and much logic runs on the server, Pixi could work – but **Phaser’s higher-level framework will likely speed up development**. Another option, if 3D was ever introduced (unlikely here), would be something like Three.js or Babylon.js, but Simulacra’s focus is 2D visuals. Given that **Phaser and React can be cleanly combined** (with modern tools like React’s state hooks managing game state, and Phaser handling rendering), the hybrid approach is confirmed as the optimal pattern for dual-mode UX ⁴ ¹⁰.

- **Responsive and Mobile Considerations:** Using React for UI also makes it easier to support different screen sizes. CSS media queries and flexbox/grid layouts can adapt the HUD and Reflection Mode layouts for smaller screens (mobile) without touching the game engine code ¹¹. Phaser can scale the canvas or use responsive scaling modes for the game view, but keeping heavy UI in HTML simplifies achieving a usable mobile experience. While desktop web is the primary target, this architecture will **gracefully accommodate mobile browsers** (Phaser games run on mobile WebGL fine, and React can adjust UI for touch input and smaller viewports). Accessibility is not a core requirement, but having critical menus and data in the DOM means features like screen readers or keyboard navigation could be added more easily later compared to a pure-canvas solution.

Conclusion: The front-end should use **Phaser 3 for the “Experience Mode” game canvas, with a React/Next.js application wrapping it for the HUD and “Reflection Mode” UI**. This hybrid pattern is widely adopted in web game development because it plays to each technology’s strengths – *fast canvas rendering for game graphics, structured DOM for interface*. As one game dev put it, “*canvas is usually better for gameplay, whereas the DOM is best for UI*” ⁷. This setup will let Simulacra deliver an engaging real-time simulation and a rich analytical toolkit in one coherent web app.

Real-Time Backend: Session Management with Colyseus vs Custom WebSockets

Recommendation: Use a **Node.js + Colyseus** server to manage multiplayer sessions and real-time communication. Colyseus is a specialized open-source multiplayer game framework that runs on Node/TypeScript, providing room management, state synchronization, and WebSocket handling out-of-the-box. This will drastically simplify the implementation of Simulacra’s real-time features (2-3 human players + AI agents per game) compared to writing a custom WebSocket server from scratch.

- **Why Colyseus:** Colyseus abstracts the low-level WebSocket work and introduces a *room/scene* paradigm that fits Simulacra’s needs exactly – each game instance can be a Colyseus “room” with its own isolated state and clients. It includes an efficient state sync mechanism where the server can define a schema for the game state and automatically send delta updates to clients. This is useful if the game state is complex and updated frequently. While Simulacra’s turn-based rounds are not extremely fast-paced (up to 30 seconds per turn), using state sync means if any small changes occur (e.g. a countdown timer tick, or partial results), they can be synced without re-sending the entire state. Developers familiar with Colyseus report that it “**gives you useful abstractions (game rooms especially)**” and is “**very quick to get up and running**” for multiplayer games ¹² ¹³. In short, it saves a lot of boilerplate: connection handling, player join/leave, messaging, and reconnection logic are largely handled by the framework. This lets the team focus on game-specific logic.
- **Colyseus Scalability:** With an MVP target of tens to a hundred concurrent games (each with ~3 players), the load on Colyseus is modest. A single Colyseus server can handle thousands of concurrent websocket connections if the game logic per room is lightweight ¹⁴ ¹⁵. Reports from the community indicate Colyseus has been used in production for cases with on the order of **1,000+ concurrent players** without issue ¹⁶. Simulacra’s CPU-intensive tasks (LLM adjudication) are offloaded to the Python service, so the Node/Colyseus server mainly handles network I/O and orchestration – which Node is very good at. We can confidently say that Colyseus on a decent cloud instance can support the MVP scale, and it provides a path to scale further (Colyseus supports clustering via a “presence server” and even offers a managed cloud service). If necessary, multiple Node instances could be run behind a load balancer, using

Colyseus's presence/redis for coordination ¹⁷, but this likely won't be needed until far beyond MVP scale.

- **Custom WebSocket + Pub/Sub Alternative:** Implementing multiplayer with a raw WebSocket library (like `ws` or Socket.io) and a pub/sub system is possible but would require significantly more engineering effort for relatively little gain at this stage. A custom approach would entail designing message protocols, handling client reconnections, ensuring only the right players receive certain data (since each player has some hidden info), and possibly coordinating state if multiple server processes are used. Colyseus already has solutions for these (e.g. **room state visibility controls** to manage per-client secret data ¹⁸). Unless the team has very unique needs not addressed by Colyseus, using it will accelerate development. The only potential downside is adding a dependency and learning its API, but the benefits (faster development and fewer bugs in networking code) outweigh that. In the Node.js community, many developers find Socket.io sufficient for simple games, but given Simulacra's requirements for authoritative state and synchronization, Colyseus's higher-level approach is well-suited. It effectively combines the familiarity of Socket.io with a built-in state sync system optimized for games ¹⁹.
- **Latency and Turn Structure:** With turn-based rounds (~5 minutes decision phase, then simultaneous resolution), the real-time demands are not extreme. Colyseus can easily handle sending out the state update at the end of each round to all players. Even if Simulacra were to incorporate some *real-time elements* (like a countdown timer ticking or a progress bar, or players selecting actions in parallel), the framework can send frequent small messages or state patches (e.g., remaining time, partial actions) without issue. The network latency (a few tens of milliseconds for WebSocket messages) is negligible compared to the 30-second round and the time spent in the LLM computation. This means we have plenty of headroom – even if in the future the game introduced more rapid interactions (say, a mode where players drag units on a map in real-time within that 30s window), Colyseus/WS could handle the updates easily.
- **Server Authoritative Architecture:** The use of Node + Colyseus also reinforces a secure, authoritative model: the server will be the source of truth for game state progression (especially since the Python engine runs there). Clients will send their chosen actions to the server, the server will validate/aggregate them and trigger the engine, then broadcast results. Colyseus naturally facilitates this server-authoritative pattern, preventing clients from cheating by modifying state locally – clients only receive what the server approves and they send input commands rather than directly manipulating state.
- **Integration with Python Engine:** Colyseus rooms can include async logic, so when a turn's timer is up or all players have locked in their actions, the room's handler can call the Python adjudication API (see next section) and await the result. During that time, the server can emit an update like "phase changing to resolution" or a loading indicator to clients via the state or messages. Once results come back, the Colyseus room updates its state and the framework will sync it to all clients. This structured flow (Room receives actions -> calls engine -> updates state -> state sync to players) maps well to Colyseus's model of iterative state updates.

In summary, **Colyseus is the right choice for the multiplayer runtime**. It provides battle-tested real-time infrastructure that aligns with Simulacra's room-based, low-frequency (per round) update model. As evidence of its suitability: *"Colyseus gives you the abstractions useful for game rooms...and is very quick to get running"*, with developers successfully using it for games with thousands of users ¹² ¹⁶. This means the team can implement multiplayer faster and with more confidence than hand-rolling socket logic. The alternative (custom WebSocket + pub/sub) might only be worth considering in the future if extremely fine-grained control or a different language/runtime is needed, but for an MVP, leveraging Colyseus will be more **MVP-friendly** and plenty **robust**.

Python Adjudication Engine: HTTP API vs gRPC/Streaming Integration

Recommendation: Continue with a **stateless Python Engine** (the “ADK”) exposed via a simple HTTP REST API (e.g. an `/adjudicate` endpoint). This service will receive the game state + player actions for a round and return the computed outcomes. For the MVP, an HTTP-based request/response model is sufficient and simpler to implement. Alternatives like gRPC or a persistent streaming connection are not necessary given the turn-based nature and relatively low call frequency, though they could be explored later if performance or real-time streaming of results becomes critical.

- **Simplicity of HTTP/REST:** Using a RESTful API call from Node to Python keeps things straightforward. Node.js can use `fetch()` or axios to POST a JSON payload of the current state and actions to the Python service, and then await a JSON response with the updated state, narrative events, score changes, etc. This approach leverages familiar web tech and JSON, which is human-readable and easy to debug. The stateless nature (each round’s adjudication is a separate call) means the Python service doesn’t need to store session data – it purely computes outputs from inputs. This scales well horizontally: if needed, multiple instances of the Python service can handle different games’ requests behind a load balancer, since any instance can adjudicate any round given the input. The current architecture already plans for this (a separate Python microservice).
- **Performance Considerations:** The overhead of an HTTP call (JSON serialization/deserialization, HTTP headers, etc.) is negligible in Simulacra’s context. Rounds happen at most once every several seconds (or minutes) per game. Even if 100 games resolved simultaneously, 100 HTTP requests is trivial for modern servers. The **bottleneck is the LLM computation** inside the Python service (which might take 1-5 seconds per round, depending on how many AI actions and narratives are generated). That dwarfs any 1-2ms difference between HTTP/1.1+JSON vs. gRPC’s binary protocol. In fact, studies show gRPC can outperform REST by up to ~7x in high-performance microservice scenarios, thanks to HTTP/2 and binary serialization ²⁰, but those gains matter more for chatty microservices or streaming; here we have relatively infrequent, large-grained calls. Using HTTP/JSON will comfortably meet the ~30s round latency budget.
- **gRPC and Streaming – Future Options:** gRPC is a strong alternative for service-to-service communication: it uses HTTP/2, binary Protobuf encoding, and can support bi-directional streaming of data. If Simulacra’s requirements evolve (e.g., wanting to stream the narrative text to players as it’s generated, rather than waiting for the whole result), a gRPC streaming response could be useful. With gRPC, the Python service could start sending partial outputs (e.g., each “beat” of the consequence narrative as it’s ready) and the Node server could forward those to clients in real-time. However, implementing this adds complexity: you’d need .proto definitions, gRPC client libraries, and to manage stream lifecycle. For MVP, this is likely over-engineering. We expect the game flow to remain turn-based, so a simple request/response fits well. Another benefit of sticking to HTTP is easier debugging and integration – the team can hit the adjudication endpoint with test tools (cURL/Postman) and inspect JSON, whereas binary gRPC requires special tooling.
- **Maintainability and Team Velocity:** The team is likely already comfortable with building REST APIs. By not introducing gRPC right now, we avoid a learning curve. Also, using JSON ensures the data structures (game state, actions, results) are human-readable, which is useful for verifying that the engine logic is working correctly (especially since LLM outputs can be complex; the team uses Zod schemas to validate them ²¹ ²²). If we used Protobuf, we’d have to define the schema in .proto and generate code – doable, but an extra step that isn’t strictly needed yet.
- **When to Consider gRPC or WebSockets:** If playtesting reveals a need for *lower latency or interactive streaming*, then we could reconsider. For example, if in Reflection Mode a user triggers

a “what-if” query that the Python engine computes on the fly, one might want a streaming update (like showing a progress or partial results). Or if the engine computations become heavy, gRPC’s efficiency could marginally help. gRPC’s use of HTTP/2 can multiplex many calls efficiently – but Node can also pipeline many async HTTP calls without blocking thanks to its event loop. Another scenario is if the Python engine needed to push spontaneous events (not in response to a request), then an event-driven approach (like the engine sending messages via a broker, or keeping a WebSocket open to the Node server) could be needed. In the current design, however, the Python service only acts when requested by the Node server; it’s a pure function from inputs to output, with no need to spontaneously initiate communication. Therefore, a synchronous HTTP call is perfectly adequate.

- **Parallel Calls and Orchestration:** During each **Consequence phase**, multiple LLM calls may happen (for AI agents’ actions, for the counterfactual, then the main consequence generation). These could be handled by the Python service in parallel. There are different ways to orchestrate this:
 - The Node server could call separate Python endpoints (e.g., `/generateActions` for each AI, concurrently, then `/adjudicate` for the main resolution) and coordinate the results.
 - Alternatively, the Python service might handle the parallelism internally (one call triggers it to internally spawn parallel tasks for AI decisions and counterfactual as described in the design ²³). Either way, the inter-service communication overhead is minor. If the Node server does multiple calls per round, using HTTP for each is fine at this scale. If that gets unwieldy, the team could consolidate into one call that does all needed computations.

In conclusion, **stick with a RESTful HTTP API for the Python engine in the MVP**. It’s easy to implement, and it aligns with the stateless, request-response nature of Simulacra’s turn cycle. We acknowledge that **gRPC is more efficient and supports streaming** (being a “high-performance, binary” RPC protocol over HTTP/2) ²⁰ and could be “ideal for high-performance applications” ²⁴ – but Simulacra is not a high-throughput microservice mesh; it’s a game with human-paced turns. The simplicity and familiarity of HTTP/JSON trump the theoretical performance gains here. Should real-time streaming of AI narratives or heavy throughput become a requirement in the future, the team can revisit this and migrate to gRPC relatively easily, but that is likely unnecessary for the foreseeable scope.

Data Persistence & Consistency: Supporting Debrief with Reliable Event Logging

Recommendation: Use **PostgreSQL as the system of record** for all game events and outcomes, and implement a careful strategy to record each round’s data in a consistent, durable way. This will involve appending to an **event log table** (or equivalent structure) every time a round resolves, capturing everything needed for the Reflection Mode – the actions taken, options available, consequences, narrative text, score changes, etc. To avoid any discrepancies between what players see in real-time and what gets saved for later analysis, adopt an atomic write-and-broadcast approach (akin to a **transactional outbox pattern**). In practice, this means the game server should **write the round results to the database *before* (or at least at the same time as) sending them to clients**, so that if an error occurs, you don’t end up with players notified of outcomes that aren’t persisted, or vice versa.

Key points for the data layer:

- **Append-Only Event Log:** As envisioned in the design, maintain an append-only history of game events. Each game will have a sequence of log entries, one per round (and possibly an initial setup entry and final summary). According to the game spec, each log entry should contain:
 - The round number (or phase).

- All player actions submitted that round (including which player did what).
- The set of available options each player had (for “action tree” visualization).
- The consequences of the joint actions: public metric changes, private score changes, narrative events described.
- The **counterfactual outcome** if no one had acted (baseline) ²⁵.

This essentially forms a *complete snapshot of decision and outcome at each turn*. The **total history H** captured this way constitutes a “complete causal graph” of the game, enabling deep post-game analysis ²⁶. In Reflection Mode, the client can use this data to replay the game step by step, draw charts of scores over time, display the decision tree of options vs choices, and allow users to explore “what-if” branches by seeing what options were not taken.

- **Data Model:** We can implement the log in Postgres either as a structured schema (multiple tables) or simply as JSON blobs per round. For an MVP, a straightforward design is:
- `games` table: one row per game (game ID, metadata, etc.).
- `game_events` table: one row per round per game, with foreign key to game ID, round index, and a JSON column containing the detail (actions, options, results, counterfactual, etc.). Using JSONB in Postgres would allow flexible schema for the event details and easy retrieval of the whole history in one query (you can aggregate JSON by game).

Alternatively, one could normalize it further (separate tables for actions, options, etc.), but given the relatively small scale (a game has at most 5 rounds in the current design, and even if that were extended, it’s not huge), a denormalized log is fine and simpler.

- **Consistency via Transactional Outbox Pattern:** The phrase “dual-write” refers to the potential problem of updating two systems (in-memory game state and the database, or database and message bus) and keeping them in sync. We want to avoid a scenario where, say, the Node server broadcasts the outcome to players but then fails to save the log to the database (e.g., DB is momentarily down or the server crashes). That would mean players saw a result that isn’t recorded for later reflection. Conversely, we don’t want to commit to the DB and then crash before sending to players – though in that case at least we have the data, but the game experience is broken. To handle this, we take inspiration from the **transactional outbox pattern**, commonly used in microservices to ensure atomicity across database and messaging ²⁷ ²⁸. In our context, the “message” is the game update to send to clients, and the database write is the log insert. The pattern suggests doing them in one transaction if possible, or transactionally writing the event, then sending. Concretely:

- When the Python engine returns the outcome for a round, the Node/Colyseus server (for that game room) should begin a database transaction.
- Insert the new event log record (or records) into Postgres (and perhaps update any aggregate fields like current scores, though those can also be derived from logs).
- Commit the transaction. Only if the commit succeeds:
- Send the outcome to clients (e.g., broadcast the new state or a message that round results are ready).

This way, we guarantee that if players see the round results, those results are safely stored in the database. If the DB write fails for some reason, we can delay or retry sending to clients (or even pause the game and show an error). Although an MVP might not need ultra-rigorous guarantees, adopting this pattern prevents edge-case data loss. As AWS’s guidance notes, the transactional outbox ensures that when an operation involves a DB update and an event notification, a failure in one won’t lead to an inconsistent system state ²⁷ ²⁹. In our case, that means avoiding the “dual-write” pitfall of updating

game state in memory and DB out-of-sync. We want **atomicity**: either both the DB log and client update happen, or neither does.

In a more advanced setup, one could even treat the database as the primary source and generate events from it (e.g., using Postgres LISTEN/NOTIFY or a message queue). But for MVP, simply controlling the order of operations in code is enough.

- **Redis for Locks/Timers:** The current plan mentions **Redis for locks and timers**. This is relevant if we have multiple Node server instances or need to handle a case where one process coordinates actions across distributed systems. For example, if we had a cluster of Node servers each running some rooms, a Redis-based lock might ensure that the adjudication for a given game is only performed once (to prevent race conditions if, say, two processes somehow attempted to resolve the same game). Similarly, for timers – if we want a robust turn timer that persists even if a server restarts – we could store a timestamp in Redis or use a Redis-based scheduler. However, if in MVP we deploy a single Node/Colyseus server, we might not need Redis at all initially; Node's in-process timers (setTimeout) can manage round countdowns. That said, as soon as we consider horizontal scaling or even just resiliency, using Redis for these ephemeral coordination tasks is wise:
- **Locks:** Ensure that only one adjudication runs per game round. We can implement this by, for instance, having a Redis key like `lock:game:{gameId}:round:{round}` that the server sets when beginning an adjudication, and releases after. In a single-process scenario, this is not needed (the room loop itself ensures one execution at a time), but with multiple processes or microservices, a distributed lock could prevent duplicate processing.
- **Timers:** If we want a turn to auto-resolve after X seconds even if no one submits (to handle slow players), the timer could be kept in Node memory. But if the Node server restarts mid-turn, that timer is lost. A Redis-based approach might store the turn deadline timestamp, and either have a separate watchdog process or each server check Redis periodically for expired timers. Another approach is to use a durable job queue or scheduler (like a delayed job in a queue) to trigger the round resolution. For MVP, this is probably overkill – Node can manage timers and, in worst-case, a game might hang if the server crashes (which is acceptable for an MVP). Over time, if reliability is needed, externalizing the timing is a consideration.
- **Scaling and Replays:** Storing a “few MB per game” in the database is no issue for PostgreSQL. 100 concurrent games, each a few MB, is just a few hundred MB of data – trivial for any modern DB. Even over time, if thousands of games are stored, we're still in the order of a few GB, which Postgres handles easily. Querying this data for Reflection Mode is straightforward: one can fetch the entire history of a game (perhaps compressed JSON) in a single query by game ID. Since our data is structured as an append-only log, it's also possible to do interesting queries (like comparing across games, or pulling out distributions of actions) if needed in the future. For now, the primary use is to retrieve a single game's log for the debrief interface. The **Reflection Mode** UI can call an API (or use Next.js server-side function) to get this data. Because all needed info is captured each round (including what options were not chosen), the reflection tools can reconstruct the decision tree and even allow hypothetical exploration. For example, the UI could let a user select an unchosen option at Round 2 and ask “what if?”. This would require re-running the Python engine from that point – which could be a future feature (the system could fork the game state at round 2 and simulate a new branch). Thanks to the stored log and the deterministic engine, this is conceptually possible. It wasn't explicitly requested for MVP, but the architecture doesn't preclude it.

- **Event Sourcing vs Simpler Logging:** Effectively, we are doing an event-sourcing-like approach, where the log of events is the source of truth and can be replayed to derive final outcomes. The game design's focus on pure functions and state immutability supports this ³⁰. For MVP, we don't necessarily need to *derive* the live game state from the database events (the game can run in memory during play and then flush to DB). But in the future, if we wanted to allow pausing/resuming games or server restarts mid-game, we could reload state by replaying the log from DB. This is a nice benefit of the approach. The **dual-write/outbox** strategy ensures that the log is complete and ordered if used for such recovery.
- **Ordering and Idempotency:** One consideration when writing events to DB and sending to clients is ensuring we don't accidentally duplicate or skip an event. The outbox pattern warns that a message might be sent twice if a failure happens at just the wrong time ³¹. Our case is simpler (we're not using a separate message broker, just direct WS sends), but we should be mindful: for example, if the server crashes after committing to DB but before sending to clients, we have an event in DB marked as last round resolved, but clients might not have gotten it. If the players reconnect and query the DB, they'd see the result, so that's actually fine (they can continue reflection, albeit the game interface would have stalled). If the server restarts, we could potentially detect that and push the result. These are edge cases likely beyond MVP scope, but it's good to note that **the consuming side (clients)** should handle receiving a duplicate event or re-fetching state gracefully. In practice, Colyseus or the game client could have a simple check (e.g., if it receives state for round 3 twice, ignore the second).

Conclusion: The data layer should be implemented with an **eye toward reliability and completeness**, since it powers the Reflection Mode features that are central to Simulacra's educational value. The design already calls for **"complete history preserved for review (append-only log)"** ³² ²⁵ – we reinforce that by storing that history in Postgres in a consistent manner. By adopting a transaction-before-broadcast rule (in spirit of the transactional outbox), we ensure that **data consistency is maintained** even in failure scenarios, preventing the classic pitfalls of dual writes ²⁷ ²⁹. This means when players enter Reflection Mode, they can trust that the data they see (replays, action trees, etc.) exactly matches what happened during gameplay.

For the MVP, implementing this could be as simple as a DB transaction per round. As the system grows, these logs could also feed other services (analytics, ML for balancing, etc.), so having them in a reliable store is invaluable. Summing up: **PostgreSQL + careful transactional logging** will give Simulacra a robust backbone for all its debriefing and analysis capabilities.

Final Verdict and MVP Implementation Guidance

After examining each layer of Simulacra's architecture, the optimal solution is largely aligned with the team's current hypothesis, with a few nuanced adjustments. The **hybrid front-end (Phaser + React), Node/TypeScript backend with Colyseus, Python LLM-driven engine via HTTP, and Postgres event logging** combination offers a highly robust yet MVP-friendly stack. It leverages well-supported technologies and patterns to meet Simulacra's unique dual-mode requirements. Below is a concise summary of recommendations, along with actionable steps for the MVP:

- **Frontend (Experience & Reflection Modes):** Proceed with the **Phaser 3 + React (Next.js) hybrid**. Use Phaser for the core simulation rendering (e.g., map, tokens, animations) in a canvas element. Overlay React components in a transparent `<div>` for UI HUD elements (score displays, timers, action selection buttons). Use standard React pages or routes for Reflection Mode screens (post-game analysis dashboards). Ensure the canvas and DOM layers

communicate – e.g., React can call Phaser APIs or vice versa via a controller – but keep them decoupled where possible. This approach is confirmed to be effective: Phaser’s own template demonstrates seamless integration with React for complex UI beyond the game world ⁴. For MVP, focus on getting the game canvas and a basic HUD working (e.g., show current round and a simple action menu in React, while Phaser perhaps highlights something on the map). Later, fill in Reflection Mode components using the logged data.

- **Game Client UX Considerations:** Keep the interface simple and clear in Experience Mode to satisfy the “general public” audience – Phaser can display intuitive graphics (e.g., icons for events or changes) while the React HUD shows minimal text prompts or choices to avoid overwhelming users. In Reflection Mode, take advantage of React’s rich ecosystem: for example, integrate a chart library for score over time, or use a tree visualization library for the action tree. Because the data is readily available (via the log), these can be implemented relatively easily once the data pipeline is in place.
- **Real-Time Multiplayer Backend:** Implement the **Colyseus server** in Node/TypeScript. Define a Room class for Simulacra games that holds the game state (at least the portion needed to sync to clients, perhaps a summary of current scores, round number, etc., since full history might not need to be synced every tick). Leverage Colyseus’s state synchronization – define a schema that includes fields each client should see. Utilize Colyseus message handlers for player actions (e.g., when a player submits an action choice, send it as a message to the room). Use the built-in timer or `setTimeout` in the room to handle the round countdown; if time expires or all actions are received, trigger the resolution. Colyseus will manage the connected clients list and reconnections. **Action:** Follow Colyseus documentation to set up the server and test a simple echo or state sync with a dummy state, then integrate it with actual game logic. The community experience suggests this will save a lot of coding vs raw sockets ¹².
- **Integration with Python Engine:** In the Colyseus room (or a controller module), implement the call to the Python service. For MVP, this can be a straightforward HTTP POST using a library like `node-fetch`. Construct the payload with all necessary info: current state (public and relevant private info), and all players’ actions for the round. Receive the response which includes the new narrative events, updated scores, etc. **Action:** Define the JSON schema for this request/response clearly (maybe reuse the Zod schemas already defined to validate the LLM output ²¹). On receiving the response, update the game state in the Colyseus room (e.g., increment round, apply new scores). Colyseus will then automatically sync those changes to clients. Make sure to handle the promise/async nature of this call so that the room waits for the result.
- **HTTP vs gRPC Decision:** Stick to **HTTP for now** – set up a simple Flask or FastAPI server in the Python engine that exposes `/adjudicate`. Ensure it can handle concurrent requests (use gunicorn or uvicorn with workers, or if using Flask’s development server, note that it might be single-threaded – better to deploy with a production WSGI server for concurrency). If needed, the Node side can fire multiple concurrent requests (for AI actions generation, etc.), but the simplest MVP path is likely one combined call per round. Document this API for clarity. Keep in mind that if performance issues arise, there is the possibility to optimize or batch work, but that likely won’t be needed initially.
- **Data Persistence & Game Logging:** Design the **Postgres schema** for game logs. Implement a function on the server side to write the log at the end of each round. Use an ORM (Prisma, TypeORM, etc.) or parameterized SQL to insert the record. **Action:** When integrating the engine result, immediately call the DB write function *before* emitting the final state to clients. Wrap this in a try/catch – if the DB write fails, you can decide to retry or signal an error. For MVP, it might be acceptable to log an error and continue (since losing one round of data in a test isn’t dire), but since Reflection Mode is a key feature, it’s better to ensure it’s saved. Testing should include verifying that after a game, all rounds are correctly saved and accessible. The log entry should closely mirror the format described in the design docs (round, actions, options, narrative, etc.)

²⁵.

- **Reflection Mode Implementation:** Build an API endpoint or use Next.js API routes to fetch the game history from Postgres. When a game ends, you might even proactively send the log to the client (or the client can call for it). For MVP, a simple approach: when the game transitions to the END phase, navigate the players (or observers) to a “Debrief” page which loads the data (by game ID). Because we have stored everything, this page can display a recap. **Action:** Implement a basic debrief page that lists rounds with the actions and outcomes (textually). This will prove that the data captured is sufficient. Later, enhance it with visualizations (e.g., a tree view where each round node expands to show the 5 options, highlighting the chosen one).
- **Critique of Current Hypothesis vs Our Plan:** The current hypothesis from the team was already on the right track:
 - Frontend hybrid (Phaser + React) – we fully agree and recommend proceeding, citing evidence that this pattern is standard and effective ⁶ ⁴ .
 - Node/TS + Colyseus – we also agree; our analysis confirms Colyseus is a strong choice for real-time sync at Simulacra’s scale, giving quick development benefits ¹³ .
 - Python engine via HTTP – we concur this is sufficient; our only note is to keep it stateless and simple for now, with the option to optimize later if needed (we don’t see a need for gRPC at MVP stage).
 - Postgres + Redis – we support Postgres as the source of truth. We advise using Redis only if needed (if a single server is used, it can be minimal at first). One area to possibly adjust: ensure that the **timers and state coordination are solid**. For MVP in a controlled environment, one Node process can handle everything (so Redis may not even be touched), but thinking ahead, the use of Redis for locks/timers will be beneficial if multi-server or to avoid Node memory single point of failure.

One addition to highlight: We strongly emphasize the **event log completeness and consistency**. The hypothesis didn’t explicitly mention how to implement consistency; our advice is to incorporate the transactional approach to avoid dual-write issues. This is a small implementation detail that can prevent potential bugs where Reflection Mode data might not match what was seen in-game if something went wrong.

- **MVP Scope and Future Evolution:** The recommended architecture is **MVP-friendly** because each piece can be built and tested in isolation and uses familiar frameworks:
 - Start with a basic Phaser game single-player (just to test Phaser+React integration).
 - Introduce Colyseus with dummy state to confirm multiplayer connectivity (e.g., a chat or simple sync).
 - Integrate the Python engine by calling a stub (before the engine is fully ready, one can simulate a response).
 - Add the database logging last, once the game loop is working, to record the outputs.
 - Build out the Reflection UI using the logged data.

This incremental approach means at each step, the team has a working system to validate (even if initially it’s a simplified game logic). The choices of Phaser, Colyseus, etc., have learning curves, but extensive documentation and community examples exist, which will aid the team in implementation. The end result will be a system where: - Users experience a smooth real-time simulation with rich graphics and interactive UI. - Expert users (or the same users, after the game) can seamlessly transition to a detailed post-game analysis powered by the data captured during play. - The backend reliably drives the game and records everything, with the AI Game Master logic cleanly encapsulated in a service that can evolve independently (e.g., upgrading the LLM, tweaking logic) without touching the core game code.

Verdict: By adhering to this architecture, Simulacra will be built on solid, modern web-gaming foundations. It maximizes development speed (using high-level frameworks) while ensuring the critical

requirements – real-time multiplayer support and rich post-game analytics – are met with reliable data and performance. Each layer’s technology was chosen based on first-principles reasoning to meet the dual experience/reflection design goals. Therefore, the team should confidently move forward with this plan. As long as best practices for each component are followed (as outlined above), the result will be a **robust MVP** that can later be scaled and refined rather than a throwaway prototype.

In summary, Simulacra’s optimal architecture is: **Phaser+React front-end, Colyseus/Node real-time server, stateless Python LLM engine via HTTP, and Postgres (with careful logging) for persistence**, optionally augmented with Redis for scalability. This ensemble addresses both the engaging “*Experience Mode*” gameplay and the insightful “*Reflection Mode*” analysis, providing a cohesive end-to-end solution for educational wargaming.

1 2 8 9 Powerful JavaScript Frameworks for Game Developers

<https://www.freecodecamp.org/news/javascript-frameworks-for-game-developers/>

3 6 7 11 5 Reasons to Use DOM Instead of Canvas for UI in HTML5 Games

<https://blog.pocketcitygame.com/5-reasons-to-use-dom-instead-of-canvas-for-ui-in-html5-games/>

4 10 Official Phaser 3 and React Template

<https://phaser.io/news/2024/02/official-phaser-3-and-react-template>

5 Canvas or DOM for HTML5 games?

<https://www.html5gamedevs.com/topic/19597-canvas-or-dom-for-html5-games/>

12 13 16 Any good multiplayer frameworks? : r/node

https://www.reddit.com/r/node/comments/1cditsy/any_good_multiplayer_frameworks/

14 15 17 18 FAQ – Colyseus

<https://docs.colyseus.io/faq>

19 Colyseus — Minimalist MMO Game Server for Node.js - Endel Dreyer

<https://endel.medium.com/colyseus-minimalistic-mmo-game-server-for-node-js-a29fe1cebbfe>

20 24 gRPC vs. REST: Key Similarities and Differences

<https://blog.dreamfactory.com/grpc-vs-rest-how-does-grpc-compare-with-traditional-rest-apis>

21 22 26 30 game-architecture.md

<https://github.com/bhi5hmaraj/ai-risk-ttx/blob/55f85d6c0444a3f7da21f2393f22cd2bdaefcdab/docs/game-architecture.md>

23 25 32 research-prompt.md

<https://github.com/bhi5hmaraj/ai-risk-ttx/blob/55f85d6c0444a3f7da21f2393f22cd2bdaefcdab/docs/research-prompt.md>

27 28 29 Transactional outbox pattern - AWS Prescriptive Guidance

<https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>

31 Pattern: Transactional outbox

<https://microservices.io/patterns/data/transactional-outbox.html>